

Análisis de capacidad

Carga de documentos:

Estas pruebas de carga fueron diseñadas para evaluar los dos flujos más importantes de la aplicación. El primer flujo abarca la etapa de *chunking* y *embedding* del documento, hasta el almacenamiento de los vectores en la base de datos vectorial. El *chunking* consiste en dividir un documento en fragmentos más pequeños (chunks) para facilitar su procesamiento y búsqueda, mientras que el *embedding* transforma cada fragmento en una representación numérica (vector) que captura su significado semántico. El segundo flujo, por su parte, corresponde a la interacción del usuario con el chat, por medio de preguntas, en función de los documentos previamente cargados.

Para poder cumplir con este fin se idearon dos escenarios:

Escenario 1 (Carga del documento):

Este escenario no posee un contexto previo, busca probar la crear credenciales y demás recursos para poder lograr la carga y almacenamiento del documento.

- Se crea un usuario, mediante el uso del endpoint de usuarios (El cual se utiliza para crear un usuario)
- Se hace login mediante el endpoint de usuarios/login, donde este me retorna el id del usuario y el token de acceso correspondiente (Importante para poder consumir los endpoints de los microservicios posteriores)
- Se crea un chat para el usuario correspondiente, mediante el uso del endpoint /chats
- Se carga un archivo a este chat mediante el uso del endpoint de chat.

Este Flujo fue creado en Jmeter, y el archivo correspondiente se llama CSV_Data_Set_prompt.jmx dentro de la carpeta Jmeter en el repositorio relacionado al proyecto (<https://github.com/leytoncasta/app-web-notebook-lm>), el cual se ve de la siguiente manera:

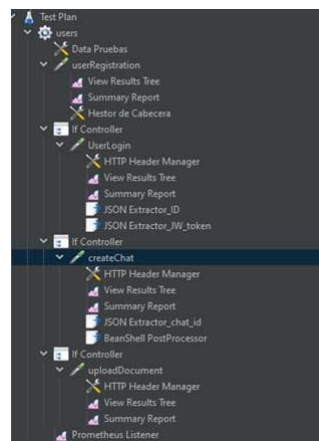


Figura 1. Pruebas Jmeter escenario 1.

Escenario 2 (Preguntas sobre el documento):

Este escenario utiliza los datos que se crearon anteriormente para la carga de documento, guardando el chat id y el token bearer que utilizaremos para la autenticación de la aplicación. Dado esto esta prueba seguirá los siguientes pasos.

- Se cargan los datos antes guardados por el escenario 1.
- Se hace una misma pregunta “¿Qué dice el texto?” para todas las threads. Todo esto al final se ve de la siguiente manera:

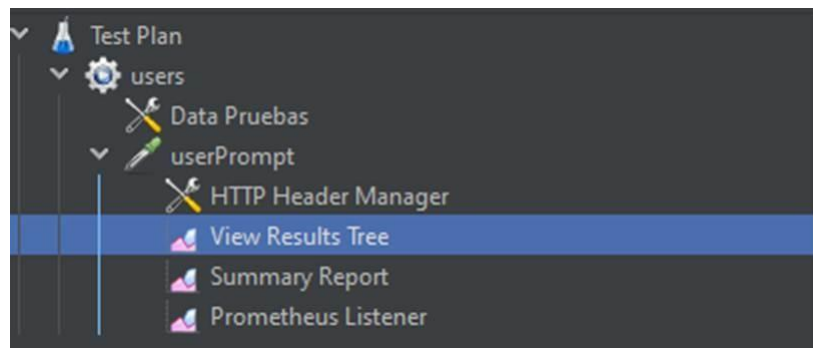


Figura 2. Pruebas Jmeter escenario 2.

Nota: A estas pruebas se añadió un simple writer para poder generar el histograma que se solicitó en la entrega pasada.

Definición de estado esperado:

Se define el estado óptimo de la aplicación bajo las siguientes condiciones:

Flujo 1

Como usuario concurrente de la aplicación, **quiero** poder completar el flujo 1 en menos de 4 segundos incluso cuando hay hasta 2 usuarios accediendo al mismo tiempo, **para** asegurar una experiencia fluida y eficiente en condiciones normales de uso.

Flujo 2

Como usuario individual de la aplicación, **quiero** recibir respuestas en menos de 1 segundo al utilizar el flujo 2, **para** garantizar una interacción rápida y continua en cualquier momento.

En dado caso que cualquiera de estas historias no se cumpla se considerara esto como un estado insuficiente.

Resultados:

Para las pruebas de carga se usó Jmeter, Prometheus y Grafana para poder observar los resultados que implican calcular tiempos de respuesta y la cantidad de solicitudes que terminaron con un status 200 o no.

Dadas estas métricas pasadas a Prometheus se calcularon las siguientes métricas utilizando estas plataformas de observabilidad:

Tasa promedio de throughput por etiqueta (ventana: 1m)

Este indicador que aparece en las gráficas como “Solicitudes exitosas al segundo en el ultimo minuto” se define como la cantidad de solicitudes con código de estado 200 procesadas por minuto por label. Para su cálculo, se configuró en Grafana una tasa (rate) de respuestas exitosas basada en la métrica denominada throughput_total. Como se muestra en la Figura 3, se utilizó el listener de Prometheus, y en la Figura 4 se presenta el código correspondiente en Grafana. Posteriormente, esta métrica fue renombrada y ahora aparece como Solicitudes exitosas por segundo en el último minuto, aunque representa el mismo valor.

```
avg by(label) (rate(throughput_total[1m]))
```

Código 1. Configuración de Grafana.

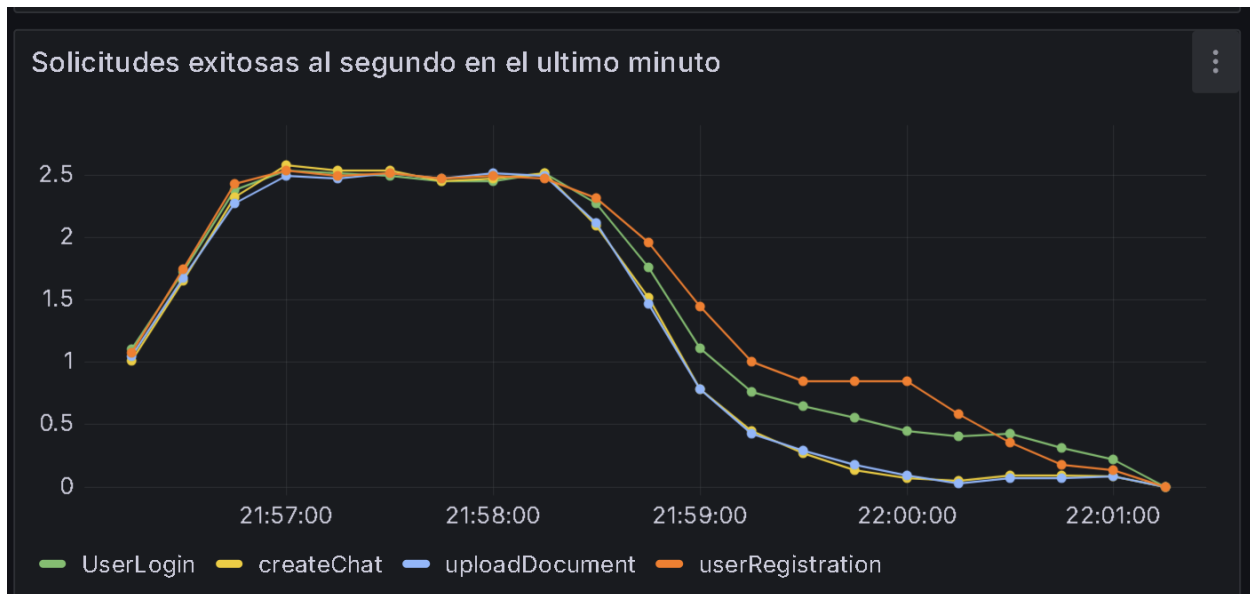


Figura 3. Tasa promedio de throughput por etiqueta (ventana: 1m)

Volumen total de eventos por segundo

Esta métrica es a grandes rasgos lo mismo que “Solicitudes exitosas al segundo generales” la diferencia está en que esta métrica es un agrupado de solicitudes agrupadas por segundo, el código se puede ver en el código 2 y la imagen en la Figura 4.

```
(sum(rate(throughput_total[1m])))
```

Código 2. Volumen total de eventos procesados por segundo en el último minuto.

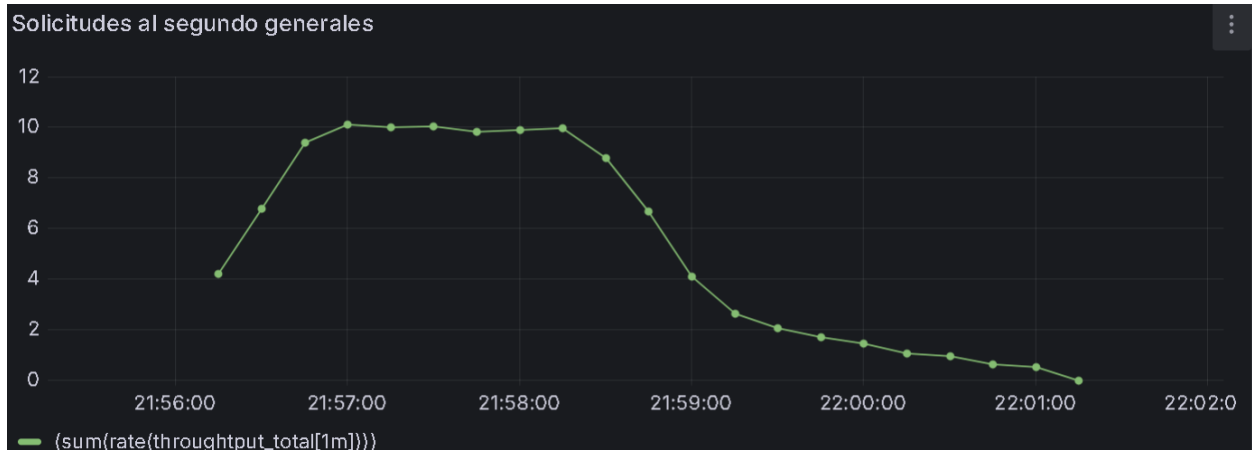


Figura 4. volumen total de eventos procesados por segundo en el último minuto.

Thread number

En la sección pasada se hizo un intento de estimación de los usuarios concurrentes de la aplicación, pero ahora ya no es necesario porque se cuenta con el número de threads en cada una de las stages (active, finished, started), para saber cuántos usuarios concurrentes tenemos al mismo tiempo haciendo las diversas solicitudes. Esto es más preciso que la métrica pasada, dado que esto nos da el número de threads precisa que finalizaron, activas e iniciando. El código de esta se puede ver en código 3 y se puede ver el tipo de grafica en la figura 5.

```
avg by (state) (avg_over_time(jmeter_threads[1m]))
```

Código 3. Configuración de Grafana de Thread number.

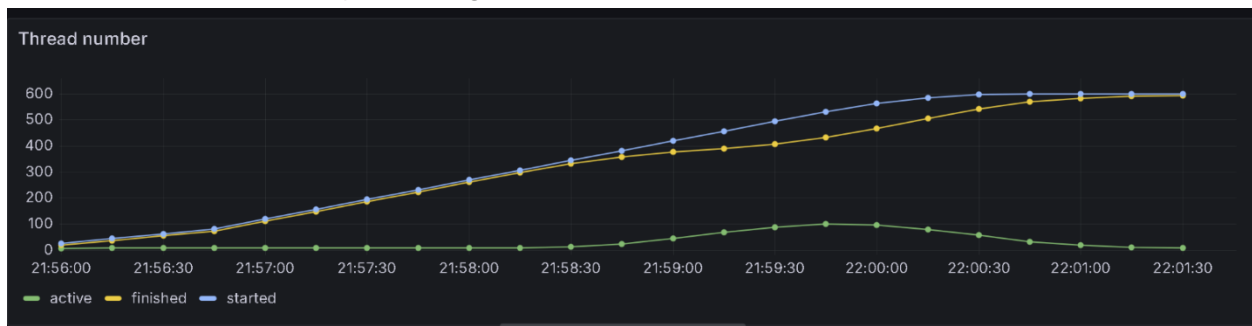


Figura 5. Configuración Grafana Thread number.

Response time:

Esta métrica abarca los tiempos de promedio por minuto de cada tipo de request. Básicamente se trata de los tiempos que se han demorado las solicitudes, dividido la cantidad de solicitudes, como se puede ver en el código 5, y su gráfica se puede ver en la *Figura 6*.

```
avg by (state) (avg_over_time(jmeter_threads[1m]))
```

Código 4. Configuración de Grafana de Response time.

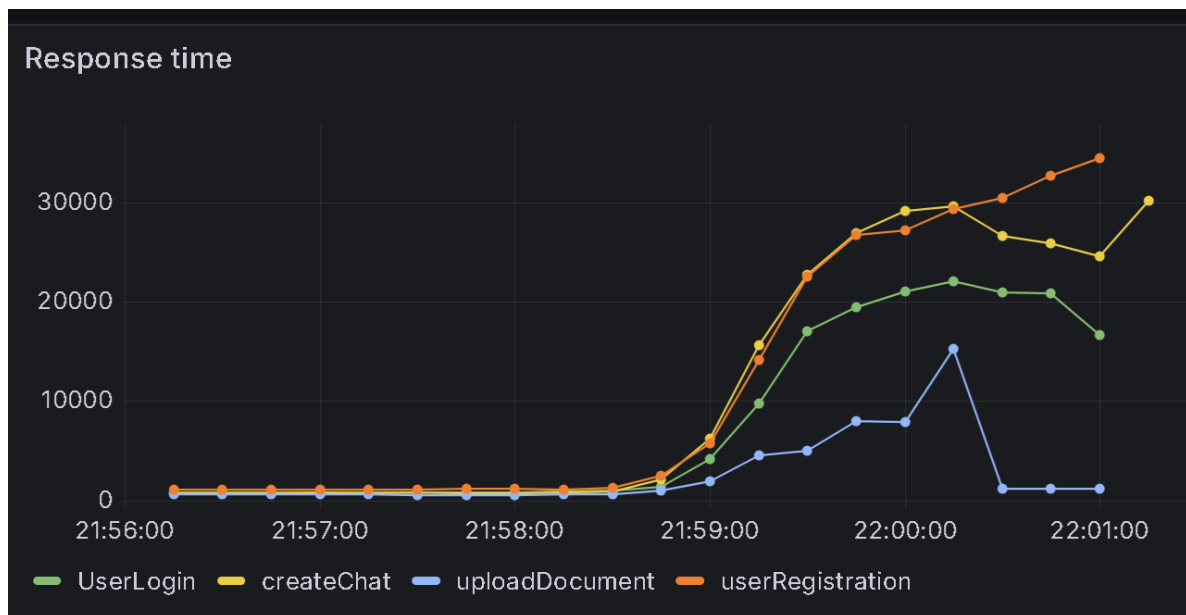


Figura 6. Tiempos de respuesta por endpoint.

Successful per Endpoint:

La métrica "Successful per endpoint" muestra cuántas veces se llamó con éxito a cada endpoint. Agrupa las solicitudes por ruta y suma cuántas se completaron bien, lo que ayuda a ver cuáles partes de tu API son más usadas. El código se puede ver en el **Código 5** y en la *figura se puede ver el resultado*.

```
sum(throughput_total) by (label)
```

Código 5. Configuración de Grafana de Response time.

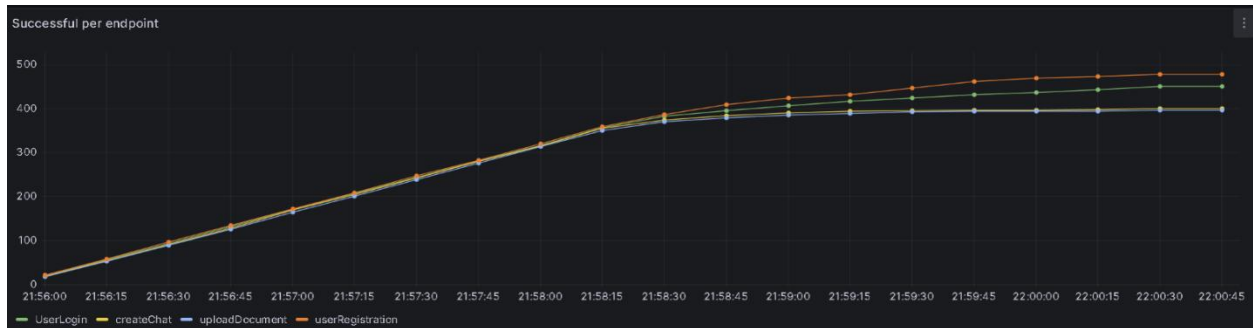


Figura 7. Grafica Successful per Endpoint.

Successful Total:

Es lo mismo que Successful per Endpoint solo que agrupado, el código se puede ver en el código 6 un ejemplo de grafica en la figura 8.

```
sum(throughput_total)
```

Código 6. Configuración de Grafana de Response time.

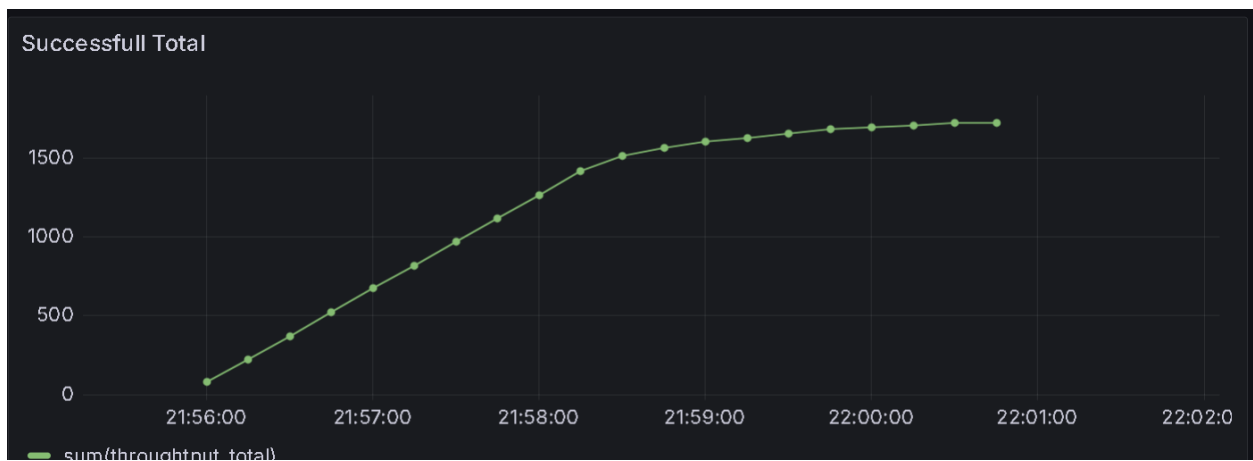


Figura 8. Grafica Successfull Total.

Response time grouped:

Esta métrica abarca los tiempos de promedio por minuto de todas las request sumadas. Básicamente se trata de los tiempos que se han demorado las solicitudes, dividido la cantidad de solicitudes, como se puede ver en el código 7, y la grafica se puede ver en la Figura 9.

```
sum(rate(response_time_summary_sum[1m])/
rate(response_time_summary_count[1m]))
```

Código 7. Configuración de Grafana de Response time.

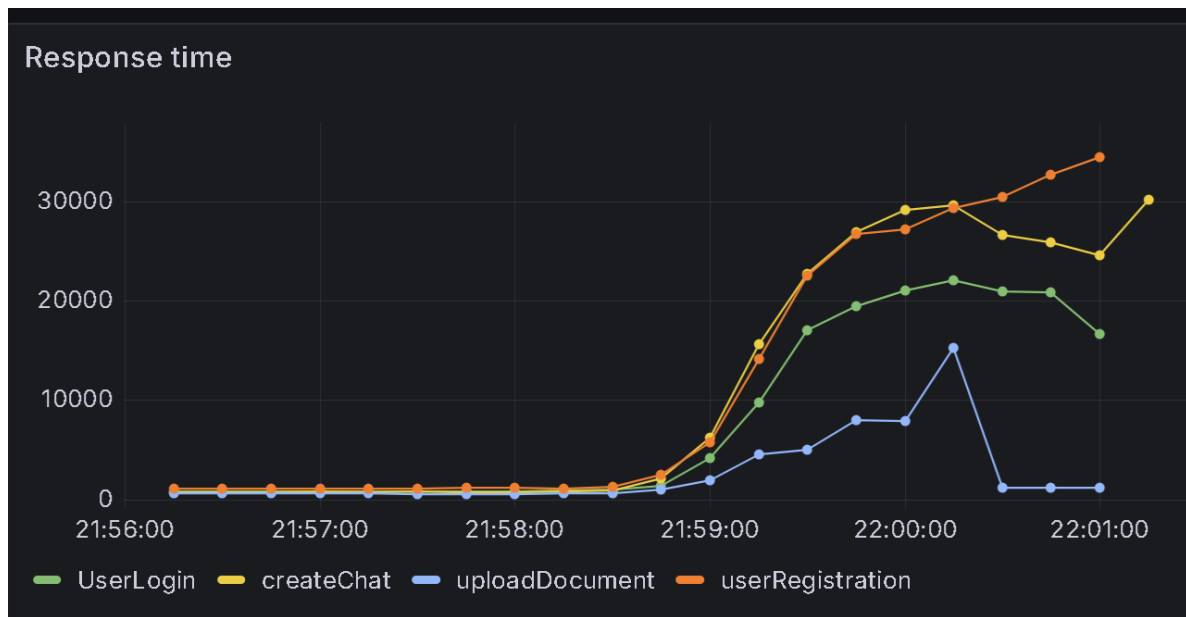


Figura 9. Tiempos de respuesta por endpoint.

Flujo final:

Se tiene que aclarar que tal como se mencionó en el documento perteneciente a la entrega de la planeación del análisis de carga (entrega pasada), se enviaron **700 hilos (threads)** distribuidos en un período de **240 segundos**, utilizando un único ciclo (*loop*). Por lo tanto, todos los resultados que se presentan a continuación corresponden a esa configuración de prueba. Se utilizó esta configuración de forma preliminar para seguir con el hilo conductor de las anteriores pruebas de carga, donde el máximo fue de 600 threads en 240 segundos, para mostrar la progresión que se ha tenido.

Resultados Escenario 1:

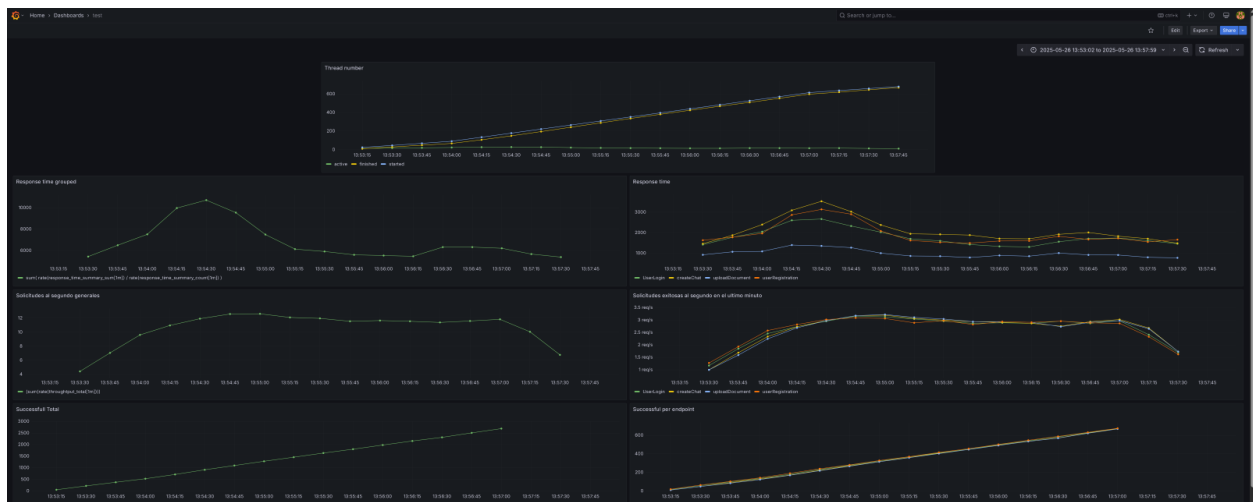


Figura 10. Tiempos de respuesta por endpoint y otras métricas.

Antes de sumergirnos en las conclusiones, queremos darles una idea general de cómo respondió el sistema durante las últimas pruebas de carga que realizamos. Enviamos un total de 700 solicitudes y nos complace ver que 669 fueron procesadas correctamente. Esto nos deja con una tasa de error de solo el 4.43%, lo cual es una mejora muy significativa en la fiabilidad si lo comparamos con versiones anteriores, sobre todo porque esta vez el sistema estaba bajo una carga más intensa.

Uno de los avances más notables lo vimos en la cantidad de usuarios que la aplicación puede atender al mismo tiempo. En el momento de mayor demanda, cerca de la 1:54 PM, llegamos a tener 29 procesos (o 'threads') funcionando en paralelo, un salto impresionante desde los 12 que estimábamos como máximo antes. Si bien es cierto que con esos 29 procesos al límite vimos algunos errores y los tiempos de respuesta se alargaron un poco, el sistema se mantuvo bastante estable. A partir de esto, calculamos que la aplicación puede manejar con soltura unas 20 solicitudes por segundo sin que se note una baja en el rendimiento, lo que definitivamente muestra cuánto ha mejorado su capacidad.

Ahora, es importante hablar de los tiempos de respuesta. Hemos notado que ahora son un poco más altos. Antes, en condiciones normales, el sistema respondía en menos de un segundo, y eso ya no sucede de forma tan consistente. Este cambio es, en realidad, un resultado directo de las mejoras que hicimos en la 'infraestructura' de la aplicación, como añadir más 'servidores' o nodos. Estos cambios la han hecho mucho más robusta y confiable, pero también han añadido un pequeño extra de tiempo en cada respuesta. Además, dado que el flujo pasa del backend a una cola, existe la posibilidad de que estos tiempos sean incluso mayores a los registrados. Sin embargo, esta consideración no contradice las conclusiones ya expuestas.

Además de esto, se tiene que la aplicación es capaz de devolver 12.6 solicitudes al segundo lo cual es mayor a la de la anterior arquitectura. Y para finalizar vemos que a pesar de la sobre carga la línea de tendencia de las solicitudes atendidas es una línea recta, dejando en claro la capacidad y estabilidad del sistema.

Para resumir, estas pruebas nos muestran que la aplicación es ahora más fuerte y puede con más usuarios y peticiones. Hemos visto menos errores y una capacidad mucho mayor para manejar trabajo simultáneo. La otra cara de la moneda es que esta mayor fiabilidad y capacidad ha venido acompañada de tiempos de respuesta un poco más largos, debido a cómo hemos reforzado su arquitectura.



Figura 11. Latencias de cada solicitud, flujo 1.

Los gráficos presentados son histogramas que muestran la latencia de cada solicitud a lo largo del tiempo. A través de ellos, se puede observar la **consistencia temporal de las métricas de latencia**, lo que permite identificar qué endpoints presentan mayor cantidad de solicitudes con alta latencia y cuán densa es dicha latencia.

Se observa que los **picos de latencia del endpoint "create chat"** superan a los de cualquier otro, lo que sugiere que, si se desea optimizar el sistema, este endpoint debería ser la primera prioridad para análisis y mejoras. Esto sigue siendo congruentes con las respuestas de la arquitectura anterior.

Pero en general a pesar de los picos que se vieron la aplicación aguanta y aunque en efecto se la tendencia al aumento de la latencia en general mantiene cierto rango permitido y optimo.

Como conclusión, y considerando tanto la magnitud como la densidad de las latencias, el orden de prioridad para posibles optimizaciones debería ser el siguiente:

1. create chat
2. user login
3. user registration
4. upload document

Aunque dado que el upload document se envía a el pubsub se espera tenga una mayor latencia.

CPU:

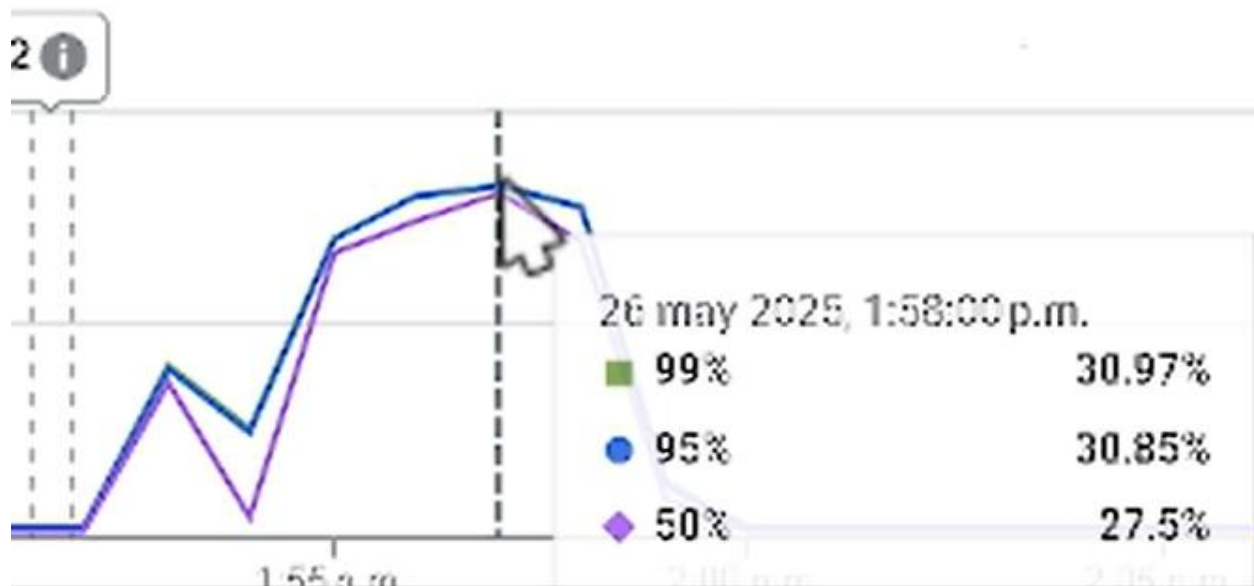


Figura 12. Consumo de CPU, del backend.

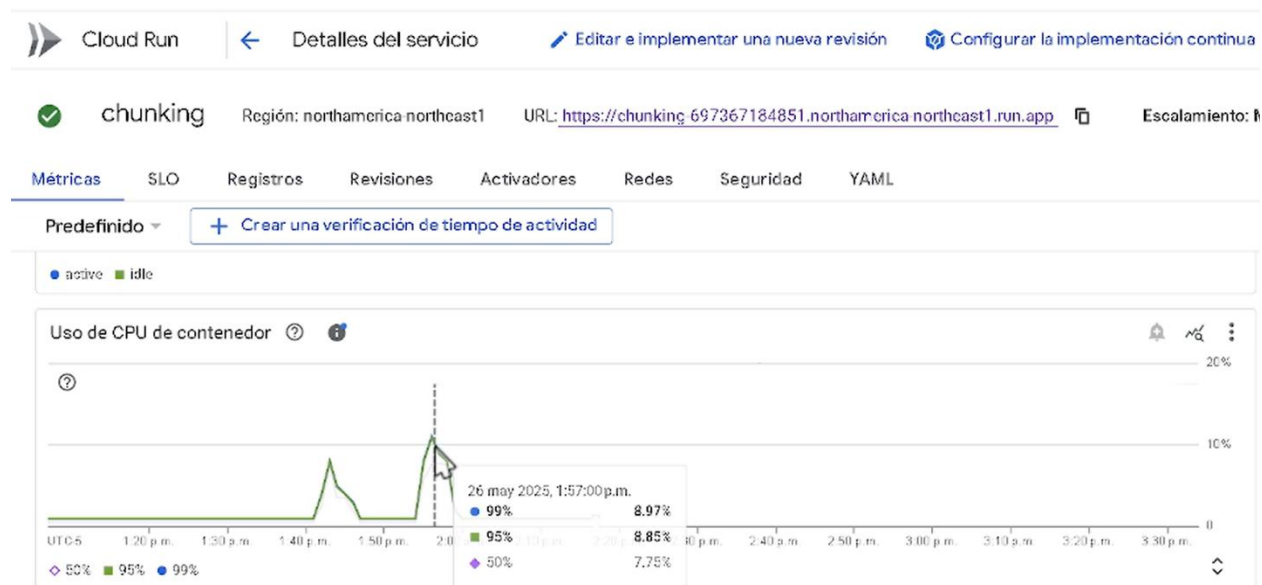


Figura 13. Consumo de CPU, del chunking.

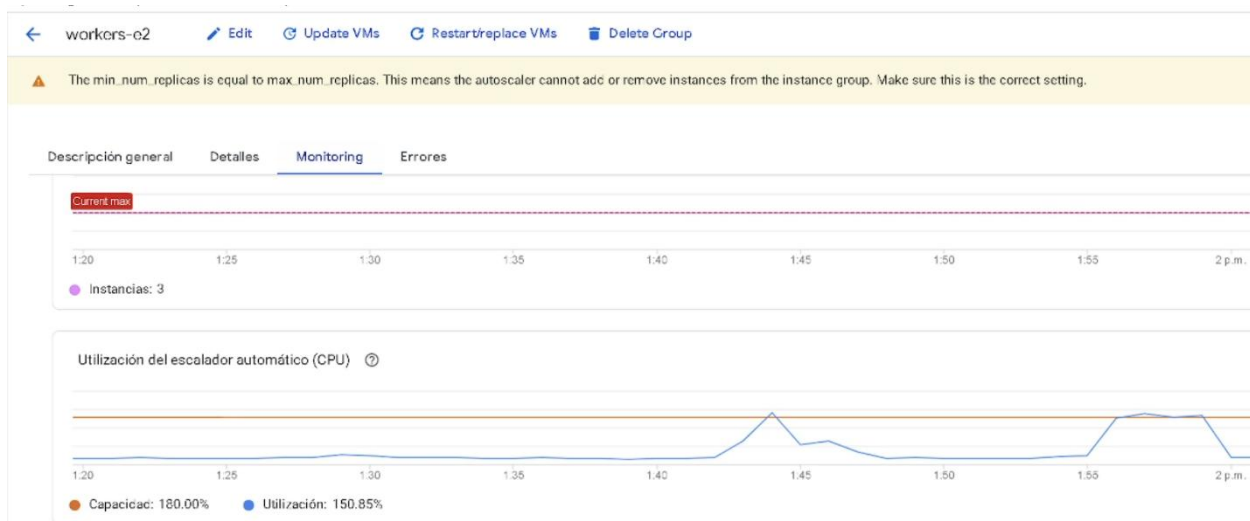


Figura 13. Consumo de CPU, de los workers.

En cuanto al uso de los procesadores (CPU), el sistema backend estuvo bastante tranquilo, usando solo hasta un 30% de su potencia. Esto nos dice que tiene mucha capacidad de sobra y no fue un problema. Cosa que también pasas con el chunking con un uso menor al 10% (es el primer pico no el segundo).

Por otro lado, el servicio encargado de los workers, que cuenta con tres máquinas para repartir el trabajo, se comportó diferente. Cada una de estas máquinas puede dar hasta un 300% de CPU (como si tuviera tres cerebros trabajando). Lo que vimos es que, en los momentos de más caña, alguna máquina individual llegó a usar un 189% de su CPU, es decir, un 63% de lo que esa máquina en particular puede dar. Aunque alguna se esforzó bastante, el hecho de que sean tres máquinas compartiendo la carga hizo que el servicio en general aguantara bien y todo siguiera funcionando sin problemas.

Resultados Escenario 2:



Figura 14. Consumo de CPU, del backend.

En esta prueba se enviaron 2400 solicitudes durante 240 segundos. Tras el pico de carga, los tiempos de respuesta mejoraron notablemente, estabilizándose en unos positivos 0.2 segundos.

Se observaron aproximadamente 3 solicitudes procesándose activamente por segundo. Esta cifra no indica baja capacidad, sino una alta eficiencia: la rápida respuesta de 0.2 segundos procesa las tareas casi de inmediato, evitando su acumulación. Si bien la arquitectura Pub/Sub puede influir en esta métrica, la velocidad de respuesta general confirma la eficiencia del sistema.

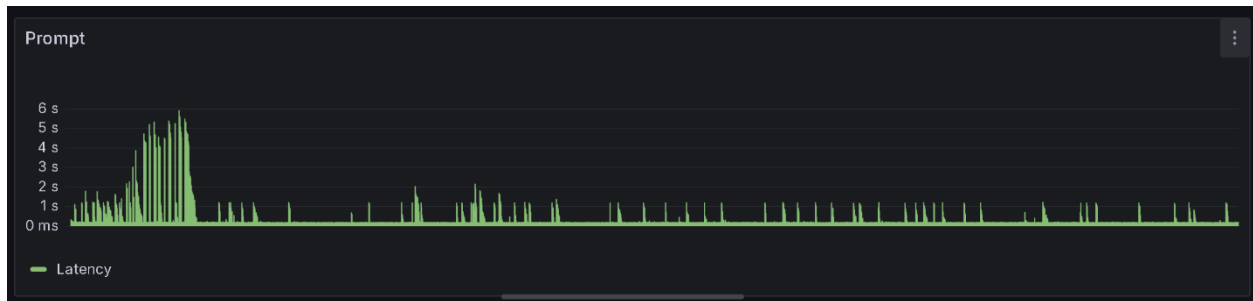


Figura 15. Latencias del prompt de forma individual.

En cuanto a los tiempos de respuesta, una vez superado el periodo inicial, estos tienden a estabilizarse, ubicándose de forma general por debajo de un segundo

CPU:



Figura 16. CPU Backend segundo flujo.

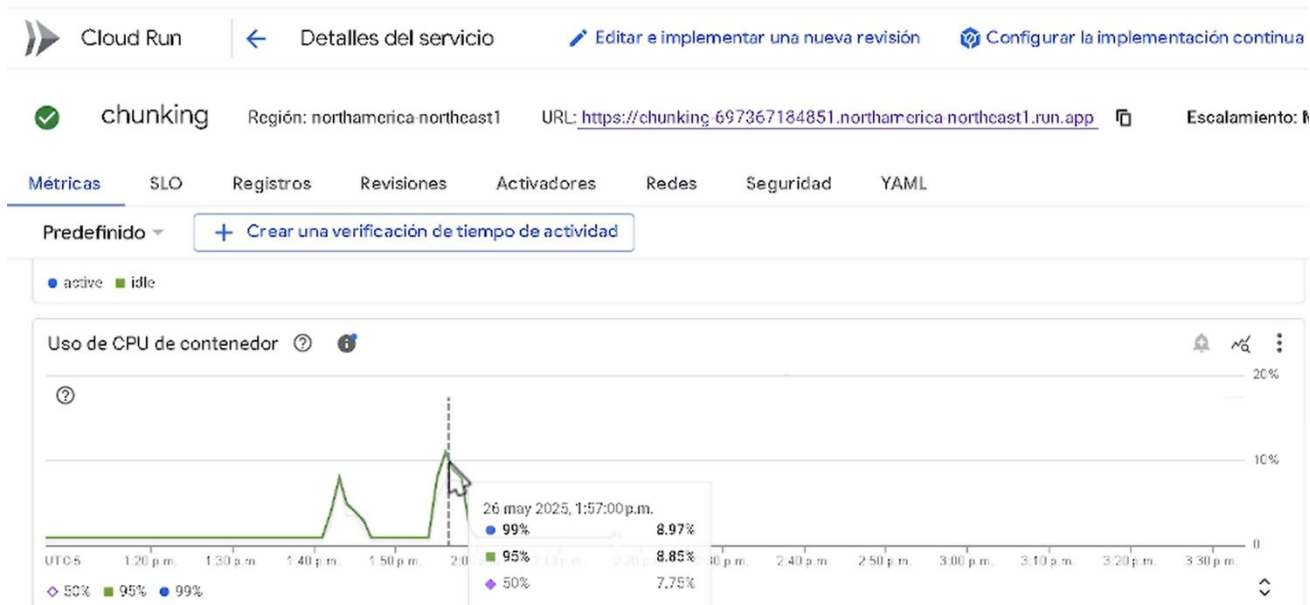


Figura 17. CPU Chunking segundo flujo.

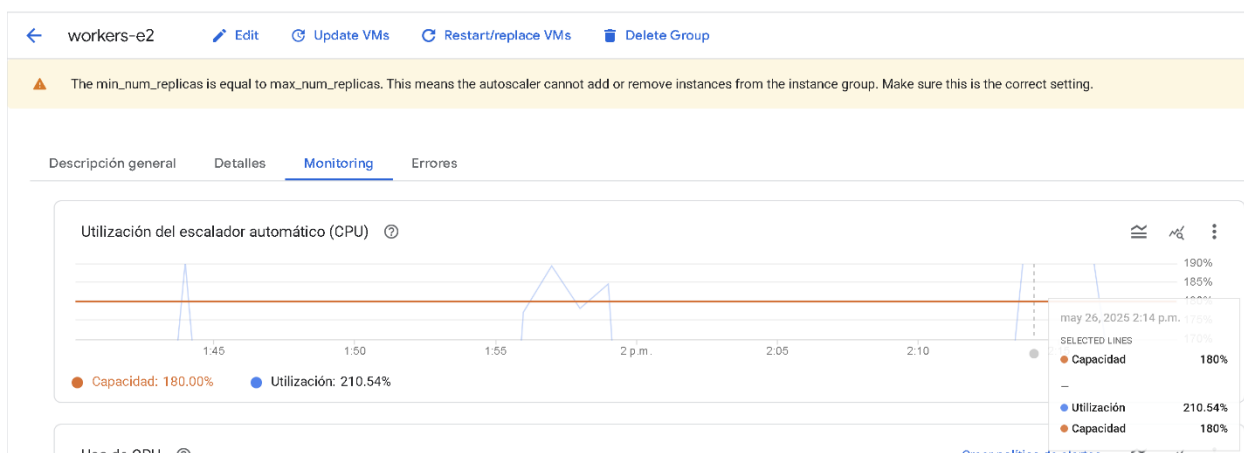


Figura 18. CPU workers segundo flujo.

Durante el análisis de carga de este flujo, entre las 2:10 PM y las 2:15 PM, la utilización de CPU del backend se mantuvo por debajo del 5%, indicando una carga mínima. De manera similar, el servicio de 'chunking' evidenció una subutilización notable, con picos de CPU por contenedor de solo el 11%. En contraste, aunque los 'workers' individuales registraron picos de uso de CPU de hasta el 210% (reflejando el uso intensivo de múltiples núcleos en instancias específicas), el sistema en su conjunto gestionó esta demanda eficazmente, demostrando su capacidad para soportar la carga requerida.

Conclusión:

Con respecto a lo que concierne a esta arquitectura, esta es mucho más robusta que la anterior, siendo capaz de soportar cargas más grandes, pero esto tiene el costo de aumentar de forma considerable las latencias, también podemos ver que la arquitectura gracias a las colas del pub sub es más robusta, y permite manejar mejor las cargas sin colapsar el sistema, tal vez por otro lado se pueda aumentar el número de mensajes que procesa cada *worker* en los *prompts*.